

Formal Verification of FiniteKernels.jl

Simon Frost

2026-09-07

Contents

FiniteKernelsProofs	1
What is formalised	1
Correspondence with the Julia API	2
FiniteKernelsProofs.Finite.Kernel	2
Conventions	2
FiniteKernelsProofs.Finite.Laws	9
FiniteKernelsProofs.Theory.Correspondence	12
Roadmap (contains sorry)	14

FiniteKernelsProofs

Lean 4 / Mathlib formalisation accompanying `FiniteKernels.jl` (toolchain `leanprover/lean4:v4.30.0`, Mathlib tag `v4.30.0`; ADR 0005). This document is generated from the Lean sources by `mdgen`: the prose is the module docstrings and the code blocks are the verbatim, machine-checked sources. Every declaration outside the final “Roadmap” section is built by `lake build --wfail` and its axioms are printed by `Audit.lean` (only `propext`, `Classical.choice`, `Quot.sound`).

What is formalised

`FiniteKernels.jl` implements finite stochastic kernels (`FiniteKernel`, tables in the outputs-first layout of ADR 0002), their sequential (`compose`) and parallel (`otimes`) composition and the copy/discard/swap structure (`mcopy`, `delete`, `braid`) as an instance of the GATlab theory `ThMarkovCategory` (SPEC §5–§6). The plan’s “Lean 4 layer” asks for the laws of that instance to be proved once, exactly, rather than only pinned numerically in `test/test_laws.jl`, and for no Mathlib `MarkovCategory` instance to be built (the open Mathlib `Stoch` pull request is not vendored). Accordingly the laws are stated pointwise about kernels.

Part	Module	Content
1	<code>Finite/Kernel.lean</code>	The finite model <code>Kernel X Y := X → Y → ℝ</code> with <code>comp</code> , <code>tensor</code> , <code>idK</code> , <code>copy</code> , <code>discard</code> , <code>swap</code> , <code>pointMass</code> , <code>ofFun</code> ; normalisation and nonnegativity are preserved; the category, symmetric-monoidal and commutative-comonoid laws.
2	<code>Finite/Laws.lean</code>	The characterisations the Julia tests pin: <code>discard</code> naturality iff normalised; <code>copy</code> naturality iff deterministic; “copy once is not two samples” for states.
3	<code>Theory/Correspondence.lean</code>	Dictionary <code>GATlab ThCopyDiscardCategory / ThMarkovCategory ↔ Mathlib CopyDiscardCategory / MarkovCategory</code> ↔ the finite model, with examples checking the type of each Mathlib field.
—	<code>Roadmap.lean</code>	<code>FinStoch</code> as a Mathlib <code>Category</code> (sorry-free) and the deferred monoidal / <code>Markov</code> instances (sorry); not in the default target.

Correspondence with the Julia API

Julia (<code>FiniteKernels.jl</code>)	Lean
<code>FiniteKernel table table[y..., x...],</code> <code>kernel_matrix(k)[y, x]</code> <code>is_normalized(k)</code> <code>compose(k, l)</code> (diagrammatic order) <code>otimes(k, l)</code> <code>id(X), mcopy(X), delete(X), braid(X, Y)</code> <code>deterministic(X, Y, f), point_mass(X, a),</code> <code>state(X, p)</code> testset “category laws” testset “monoidal laws”	<code>Kernel X Y, entry k x y = P(y x)</code> <code>Kernel.Normalised k</code> <code>Kernel.comp k l</code> <code>Kernel.tensor k l</code> <code>idK X, copy X, discard X, swap X Y</code> <code>ofFun f, pointMass a, State X</code> <code>comp_assoc, idK_comp, comp_idK</code> <code>comp_tensor, tensor_idK, tensor_assoc,</code> <code>tensor_unit_left, tensor_unit_right, swap_swap,</code> <code>tensor_swap, hexagon</code> <code>copy_assoc, copy_discard_left, copy_discard_right,</code> <code>copy_swap, copy_prod, discard_prod, copy_unit,</code> <code>discard_unit</code> <code>comp_discard_eq_discard_iff</code> <code>comp_copy_eq_iff_isDeterministic</code> <code>comp_copy_eq_tensor_iff_isPointMass</code>
testsets “comonoid laws”, “coherence of copy and discard”	
“discard naturality iff normalised”	
“copy is natural for deterministic kernels only”	
“copy once is not two samples” (SPEC §6)	

SPEC §61’s Propositions 1–7 concern Bayesian networks and influence diagrams and are formalised in `BayesianNetworks.jl/proofs` and `InfluenceDiagrams.jl/proofs`; this project supplies the kernel-level laws those build on. The generic consequences of the abstract axioms (`discard_natural` as a theorem, `state_discard`, `deterministic_comp`, `deterministic_copy`) are stated once, for Mathlib’s classes, in `BayesianNetworksProofs.Markov.Basic`.

```
namespace FiniteKernelsProofs

/-- Smoke lemma so that the axiom audit always has a first line. -/
theorem smoke : (1 : Nat) + 1 = 2 := rfl

end FiniteKernelsProofs
```

FiniteKernelsProofs.Finite.Kernel

```
import Mathlib.Data.Real.Basic
import Mathlib.Algebra.BigOperators.Ring.Finset
import Mathlib.Data.Fintype.BigOperators
import Mathlib.Data.Fintype.Prod
import Mathlib.Logic.Equiv.Basic
import Mathlib.Algebra.Order.BigOperators.Group.Finset
import Mathlib.Tactic.Ring
```

A concrete finite model of the structure implemented by `FiniteKernels.jl` (`src/kernels.jl`, `src/composition.jl`, `src/copy_discard.jl`): finite stochastic kernels between finite types, with sequential composition, tensor product, copy, discard and swap.

No category instance is built here (ADR 0005, plan “Lean 4 layer”); the laws are stated directly about kernels, pointwise, with the monoidal structure maps (associator, unitors, $\text{tensor}\mu$) written as *function-reindexing kernels* `ofFun e` for the obvious equivalences.

Conventions

- `Kernel X Y := X → Y → ℝ`; $k \ x \ y$ is the Julia entry $P(y \mid x)$, i.e. `table[y..., x...]` in the outputs-first layout of ADR 0002 (`kernel_matrix(k)[y, x]`).
- `Normalised k` is `is_normalized(k)`: every column $k \ x$ sums to one.

- Entries live in $\mathbb{R}$ rather than $\mathbb{R}_{\geq 0}$; nonnegativity is the separate predicate `Nonneg` and is preserved by every construction, but none of the laws below need it.
- `comp k l` is the diagrammatic composite `compose(k, l)` (first `k`, then `l`), i.e. the matrix product `kernel_matrix(l) * kernel_matrix(k)`.
- `tensor k l` is `otimes(k, l)`, the outer product of the tables.
- Deterministic kernels are `ofFun f` (Julia `deterministic(X, Y, f)`); the identity, copy, discard, swap and point masses are all of this form, which is what makes the structural laws one-line proofs.

```
namespace FiniteKernelsProofs.Finite

open Finset

/-- A finite kernel 'X → Y' is a real-valued table 'k x y = P(y | x)' ('FiniteKernel.table' in
the outputs-first layout, read as a function of the input first). -/
abbrev Kernel (X Y : Type) : Type := X → Y → ℝ

/-- A state 'I → X' ('state(X, probs)'): a kernel out of the monoidal unit 'Unit'. -/
abbrev State (X : Type) : Type := Kernel Unit X

namespace Kernel

variable {X Y Z W X₁ X₂ X₃ Y₁ Y₂ Y₃ Z₁ Z₂ : Type}
```

Predicates

```
/-- 'is_normalized(k)': each column sums to one. -/
def Normalised [Fintype Y] (k : Kernel X Y) : Prop := ∀ x, ∑ y, k x y = 1

/-- All entries are nonnegative. -/
def Nonneg (k : Kernel X Y) : Prop := ∀ x y, 0 ≤ k x y

/-- A stochastic kernel: nonnegative and normalised (a morphism of **FinStoch**). -/
def Stochastic [Fintype Y] (k : Kernel X Y) : Prop := Nonneg k ∧ Normalised k

/-- A deterministic kernel in the sense of the Julia tests: every column is a point mass. -/
def IsDeterministic [DecidableEq Y] (k : Kernel X Y) : Prop :=
  ∀ x, ∃ y, ∀ y', k x y' = if y' = y then 1 else 0

/-- A state is a point mass ('point_mass(X, a)') at some 'a'. -/
def IsPointMass [DecidableEq X] (p : State X) : Prop :=
  ∃ a, ∀ x, p () x = if x = a then 1 else 0
```

Constructions

```
/-- Sequential composition 'compose(k, l)' (diagrammatic order): '(k • l)(z | x) = ∑ y k(y | x) l(z |
  ↪ y)',
the matrix product 'kernel_matrix(l) * kernel_matrix(k)'. -/
def comp [Fintype Y] (k : Kernel X Y) (l : Kernel Y Z) : Kernel X Z :=
  fun x z => ∑ y, k x y * l y z

/-- Parallel composition 'otimes(k, l)': the outer product '(k ⊗ l)(y₁, y₂ | x₁, x₂) = k(y₁ | x₁)
  ↪ l(y₂ | x₂)'. -/
def tensor (k : Kernel X₁ Y₁) (l : Kernel X₂ Y₂) : Kernel (X₁ × X₂) (Y₁ × Y₂) :=
  fun x y => k x.1 y.1 * l x.2 y.2

/-- The deterministic kernel 'deterministic(X, Y, f)' induced by a function: 'P(y | x) = [y = f x]'.
  ↪ -/
def ofFun [DecidableEq Y] (f : X → Y) : Kernel X Y :=
  fun x y => if y = f x then 1 else 0

/-- The identity kernel 'id(X)' ('identity_kernel'). -/
def idK (X : Type) [DecidableEq X] : Kernel X X := ofFun id
```

```

/-- The copy map `mcopy(X) = Δ(X) : X → X ⊗ X`, `Δ(x₁, x₂ | x) = [x₁ = x][x₂ = x]` ('copy_kernel').
  ↪ -/
def copy (X : Type) [DecidableEq X] : Kernel X (X × X) := ofFun fun x ⇒ (x, x)

/-- The discard map `delete(X) = ◊(X) : X → I`, the table of ones ('discard_kernel'). -/
def discard (X : Type) : Kernel X Unit := fun _ _ ⇒ 1

/-- The symmetry `braid(X, Y) = σ(X, Y) : X ⊗ Y → Y ⊗ X` ('swap_kernel'). -/
def swap (X Y : Type) [DecidableEq X] [DecidableEq Y] : Kernel (X × Y) (Y × X) :=
  ofFun (Equiv.prodComm X Y)

/-- The point mass `point_mass(X, a) = dirac(X, a) : I → X`. -/
def pointMass [DecidableEq X] (a : X) : State X := ofFun fun _ ⇒ a

/-- Push a state through a kernel (`apply(k, p) = compose(p, k)`). -/
def push [Fintype X] (k : Kernel X Y) (p : State X) : State Y := comp p k

```

Structural reindexing kernels (the monoidal coherence data).

FiniteKernels.jl is strict (Julia tuples are flat), so these are all identities there; Mathlib's MonoidalCategory is not strict, and its fields refer to them by name.

```

/-- The associator `α_X Y Z : (X ⊗ Y) ⊗ Z → X ⊗ (Y ⊗ Z)` as a kernel. -/
def assoc (X Y Z : Type) [DecidableEq X] [DecidableEq Y] [DecidableEq Z] :
  Kernel ((X × Y) × Z) (X × (Y × Z)) := ofFun (Equiv.prodAssoc X Y Z)

/-- The left unitor `λ_X : I ⊗ X → X` as a kernel. -/
def leftUnitor (X : Type) [DecidableEq X] : Kernel (Unit × X) X := ofFun (Equiv.punitProd X)

/-- The inverse left unitor `(λ_X).inv : X → I ⊗ X` as a kernel. -/
def leftUnitorInv (X : Type) [DecidableEq X] : Kernel X (Unit × X) := ofFun (Equiv.punitProd X).symm

/-- The right unitor `ρ_X : X ⊗ I → X` as a kernel. -/
def rightUnitor (X : Type) [DecidableEq X] : Kernel (X × Unit) X := ofFun (Equiv.prodPUnit X)

/-- The inverse right unitor `(ρ_X).inv : X → X ⊗ I` as a kernel. -/
def rightUnitorInv (X : Type) [DecidableEq X] : Kernel X (X × Unit) := ofFun (Equiv.prodPUnit X).symm

/-- Mathlib's `tensorμ X X Y Y : (X ⊗ X) ⊗ (Y ⊗ Y) → (X ⊗ Y) ⊗ (X ⊗ Y)`, the middle swap
  `id(X) ⊗ σ(X, Y) ⊗ id(Y)` of Catlab's coherence axiom for `mcopy`. -/
def tensorμ (X Y : Type) [DecidableEq X] [DecidableEq Y] :
  Kernel ((X × X) × (Y × Y)) ((X × Y) × (X × Y)) :=
  ofFun fun p ⇒ ((p.1.1, p.2.1), (p.1.2, p.2.2))

/-- The unique map `I ⊗ I → I` (`(λ_I).hom`). -/
def unitMul : Kernel (Unit × Unit) Unit := ofFun fun _ ⇒ ()

```

Pointwise evaluation lemmas

```

theorem ext_iff {k l : Kernel X Y} : k = l ↔ ∀ x y, k x y = l x y := by
  simp [funext_iff]

@[simp] theorem comp_apply [Fintype Y] (k : Kernel X Y) (l : Kernel Y Z) (x : X) (z : Z) :
  comp k l x z = ∑ y, k x y * l y z := rfl

@[simp] theorem tensor_apply (k : Kernel X₁ Y₁) (l : Kernel X₂ Y₂) (x : X₁ × X₂) (y : Y₁ × Y₂) :
  tensor k l x y = k x.1 y.1 * l x.2 y.2 := rfl

@[simp] theorem ofFun_apply [DecidableEq Y] (f : X → Y) (x : X) (y : Y) :
  ofFun f x y = if y = f x then 1 else 0 := rfl

@[simp] theorem discard_apply (x : X) (u : Unit) : discard X x u = 1 := rfl

theorem idK_apply [DecidableEq X] (x y : X) : idK X x y = if y = x then 1 else 0 := rfl

```

```

theorem copy_apply [DecidableEq X] (x x1 x2 : X) :
  copy X x (x1, x2) = if x1 = x ∧ x2 = x then 1 else 0 := by
  simp [copy, Prod.ext_iff]

theorem swap_apply [DecidableEq X] [DecidableEq Y] (x : X) (y : Y) (y' : Y) (x' : X) :
  swap X Y (x, y) (y', x') = if y' = y ∧ x' = x then 1 else 0 := by
  simp [swap, Prod.ext_iff]

theorem pointMass_apply [DecidableEq X] (a x : X) : pointMass a () x = if x = a then 1 else 0 := rfl

/-- 'discard' is the deterministic kernel of the unique map to 'Unit'. -/
theorem discard_eq_ofFun : discard X = ofFun fun _ => () := by
  funext x u; simp

```

The deterministic-kernel calculus

Every structural morphism is ofFun $_$, and ofFun is a functor from functions to kernels that is compatible with tensor. Together with the two lemmas for composing an arbitrary kernel with ofFun on either side, these make the monoidal and comonoid laws mechanical.

```

/-- Precomposing with a deterministic kernel reindexes the input. -/
theorem comp_ofFun_left [Fintype Y] [DecidableEq Y] (f : X → Y) (l : Kernel Y Z) :
  comp (ofFun f) l = fun x z => l (f x) z := by
  funext x z
  simp [comp, ite_mul, Finset.sum_ite_eq']

/-- Postcomposing with the deterministic kernel of an equivalence reindexes the output. -/
theorem comp_ofEquiv_right [Fintype Y] [DecidableEq Z] (k : Kernel X Y) (e : Y ≃ Z) :
  comp k (ofFun e) = fun x z => k x (e.symm z) := by
  funext x z
  simp only [comp_apply, ofFun_apply, mul_ite, mul_one, mul_zero]
  rw [Finset.sum_eq_single (e.symm z)]
  · simp
  · intro b _ hb
    rw [if_neg]
    exact fun h => hb (e.symm_apply_eq.mpr h).symm
  · simp

/-- Postcomposing with a general deterministic kernel pushes mass forward. -/
theorem comp_ofFun_right [Fintype Y] [DecidableEq Z] (k : Kernel X Y) (g : Y → Z) :
  comp k (ofFun g) = fun x z => ∑ y, if z = g y then k x y else 0 := by
  funext x z
  simp [comp]

theorem ofFun_comp_ofFun [Fintype Y] [DecidableEq Y] [DecidableEq Z] (f : X → Y) (g : Y → Z) :
  comp (ofFun f) (ofFun g) = ofFun (g ∘ f) := by
  rw [comp_ofFun_left]; rfl

theorem tensor_ofFun [DecidableEq Y1] [DecidableEq Y2] (f : X1 → Y1) (g : X2 → Y2) :
  tensor (ofFun f) (ofFun g) = ofFun (Prod.map f g) := by
  funext x y
  rcases x with ⟨x1, x2⟩; rcases y with ⟨y1, y2⟩
  simp only [tensor_apply, ofFun_apply, Prod.map_apply, Prod.mk.injEq]
  split_ifs <|> simp_all

theorem ofFun_congr [DecidableEq Y] {f g : X → Y} (h : ∀ x, f x = g x) : ofFun f = ofFun g := by
  rw [funext h]

```

Normalisation and nonnegativity are preserved

```

theorem Normalised.comp [Fintype Y] [Fintype Z] {k : Kernel X Y} {l : Kernel Y Z}
  (hk : Normalised k) (hl : Normalised l) : Normalised (comp k l) := by
  intro x

```

```

have hl' : ∀ y, ∑ z, l y z = 1 := hl
simp only [comp_apply]
rw [Finset.sum_comm]
simp [← Finset.mul_sum, hl']
exact hk x

theorem Normalised.tensor [Fintype Y1] [Fintype Y2] {k : Kernel X1 Y1} {l : Kernel X2 Y2}
  (hk : Normalised k) (hl : Normalised l) : Normalised (tensor k l) := by
  intro x
  simp only [tensor_apply]
  rw [Fintype.sum_prod_type, ← Finset.sum_mul_sum, hk, hl, one_mul]

theorem Normalised.ofFun [Fintype Y] [DecidableEq Y] (f : X → Y) : Normalised (ofFun f) := by
  intro x
  simp [Finset.sum_ite_eq']

theorem Normalised.idK [Fintype X] [DecidableEq X] : Normalised (idK X) := Normalised.ofFun _
theorem Normalised.copy [Fintype X] [DecidableEq X] : Normalised (copy X) := Normalised.ofFun _
theorem Normalised.discard : Normalised (discard X) := by intro x; simp
theorem Normalised.swap [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y] :
  Normalised (swap X Y) := Normalised.ofFun _
theorem Normalised.pointMass [Fintype X] [DecidableEq X] (a : X) : Normalised (pointMass a) :=
  Normalised.ofFun _

theorem Nonneg.comp [Fintype Y] {k : Kernel X Y} {l : Kernel Y Z} (hk : Nonneg k) (hl : Nonneg l) :
  Nonneg (comp k l) := fun x z =>
  Finset.sum_nonneg fun y _ => mul_nonneg (hk x y) (hl y z)

theorem Nonneg.tensor {k : Kernel X1 Y1} {l : Kernel X2 Y2} (hk : Nonneg k) (hl : Nonneg l) :
  Nonneg (tensor k l) := fun _ _ => mul_nonneg (hk _ _) (hl _ _)

theorem Nonneg.ofFun [DecidableEq Y] (f : X → Y) : Nonneg (ofFun f) := by
  intro _ _; simp only [ofFun_apply]; split_ifs <=> norm_num

theorem Nonneg.discard : Nonneg (discard X) := fun _ _ => zero_le_one

theorem Stochastic.comp [Fintype Y] [Fintype Z] {k : Kernel X Y} {l : Kernel Y Z}
  (hk : Stochastic k) (hl : Stochastic l) : Stochastic (comp k l) :=
  ⟨hk.1.comp hl.1, hk.2.comp hl.2⟩

theorem Stochastic.tensor [Fintype Y1] [Fintype Y2] {k : Kernel X1 Y1} {l : Kernel X2 Y2}
  (hk : Stochastic k) (hl : Stochastic l) : Stochastic (tensor k l) :=
  ⟨hk.1.tensor hl.1, hk.2.tensor hl.2⟩

theorem Stochastic.ofFun [Fintype Y] [DecidableEq Y] (f : X → Y) : Stochastic (ofFun f) :=
  ⟨Nonneg.ofFun f, Normalised.ofFun f⟩

```

Category laws (Julia testset “category laws”)

```

theorem comp_assoc [Fintype Y] [Fintype Z] (k : Kernel X Y) (l : Kernel Y Z) (m : Kernel Z W) :
  comp (comp k l) m = comp k (comp l m) := by
  funext x w
  simp only [comp_apply, Finset.sum_mul, Finset.mul_sum]
  rw [Finset.sum_comm]
  simp [mul_assoc]

theorem idK_comp [Fintype X] [DecidableEq X] (k : Kernel X Y) : comp (idK X) k = k := by
  rw [idK, comp_ofFun_left]; rfl

theorem comp_idK [Fintype Y] [DecidableEq Y] (k : Kernel X Y) : comp k (idK Y) = k := by
  rw [idK]; exact comp_ofEquiv_right k (Equiv.refl Y)

```

Monoidal laws (Julia testset “monoidal laws”)

```

/-- Interchange: 'compose(otimes(k, l), otimes(k', l')) = otimes(compose(k, k'), compose(l, l'))'. -/
theorem comp_tensor [Fintype Y1] [Fintype Y2] (k : Kernel X1 Y1) (l : Kernel X2 Y2)
  (k' : Kernel Y1 Z1) (l' : Kernel Y2 Z2) :
  comp (tensor k l) (tensor k' l') = tensor (comp k k') (comp l l') := by
  funext x z
  simp only [comp_apply, tensor_apply, Fintype.sum_prod_type, Finset.sum_mul_sum]
  exact Finset.sum_congr rfl fun y1 _ => Finset.sum_congr rfl fun y2 _ => by ring

/-- 'otimes(id(X), id(Y)) = id(otimes(X, Y))'. -/
theorem tensor_idK [DecidableEq X] [DecidableEq Y] : tensor (idK X) (idK Y) = idK (X × Y) := by
  simp only [idK, tensor_ofFun]; rfl

/-- Associativity of the tensor, up to the associator:
'otimes(otimes(k, l), m) · α = α · otimes(k, otimes(l, m))'. -/
theorem tensor_assoc [Fintype X1] [Fintype X2] [Fintype X3] [Fintype Y1] [Fintype Y2] [Fintype Y3]
  [DecidableEq X1] [DecidableEq X2] [DecidableEq X3] [DecidableEq Y1] [DecidableEq Y2]
  [DecidableEq Y3] (k : Kernel X1 Y1) (l : Kernel X2 Y2) (m : Kernel X3 Y3) :
  comp (tensor (tensor k l) m) (assoc Y1 Y2 Y3) = comp (assoc X1 X2 X3) (tensor k (tensor l m)) :=
  by
  rw [assoc, assoc, comp_ofEquiv_right, comp_ofFun_left]
  funext x y
  rcases x with ⟨⟨x1, x2⟩, x3⟩; rcases y with ⟨y1, y2, y3⟩
  simp [mul_assoc]

/-- Right unit of the tensor, up to the right unitor: 'otimes(k, id(I)) · ρ = ρ · k'. -/
theorem tensor_unit_right [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y] (k : Kernel X Y) :
  comp (tensor k (idK Unit)) (rightUnitor Y) = comp (rightUnitor X) k := by
  rw [rightUnitor, rightUnitor, comp_ofEquiv_right, comp_ofFun_left]
  funext x y
  simp [idK]

/-- Left unit of the tensor, up to the left unitor: 'otimes(id(I), k) · λ = λ · k'. -/
theorem tensor_unit_left [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y] (k : Kernel X Y) :
  comp (tensor (idK Unit) k) (leftUnitor Y) = comp (leftUnitor X) k := by
  rw [leftUnitor, leftUnitor, comp_ofEquiv_right, comp_ofFun_left]
  funext x y
  simp [idK]

/-- The symmetry is an involution: 'braid(X, Y) · braid(Y, X) = id(X ⊗ Y)'. -/
theorem swap_swap [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y] :
  comp (swap X Y) (swap Y X) = idK (X × Y) := by
  rw [swap, swap, ofFun_comp_ofFun, idK]; rfl

/-- Naturality of the symmetry: 'otimes(k, l) · braid = braid · otimes(l, k)'. -/
theorem tensor_swap [Fintype X1] [Fintype X2] [Fintype Y1] [Fintype Y2] [DecidableEq X1]
  [DecidableEq X2] [DecidableEq Y1] [DecidableEq Y2] (k : Kernel X1 Y1) (l : Kernel X2 Y2) :
  comp (tensor k l) (swap Y1 Y2) = comp (swap X1 X2) (tensor l k) := by
  rw [swap, swap, comp_ofEquiv_right, comp_ofFun_left]
  funext x y
  simp [mul_comm]

/-- The hexagon identity ('SymmetricCategory'/'BraidedCategory.hexagon_forward'), which is the
associator-explicit form of the Julia test
'braid(X ⊗ Y, Z) = (id(X) ⊗ braid(Y, Z)) · (braid(X, Z) ⊗ id(Y))'. -/
theorem hexagon [Fintype X] [Fintype Y] [Fintype Z] [DecidableEq X] [DecidableEq Y]
  [DecidableEq Z] :
  comp (assoc X Y Z) (comp (swap X (Y × Z)) (assoc Y Z X)) =
  comp (tensor (swap X Y) (idK Z)) (comp (assoc Y X Z) (tensor (idK Y) (swap X Z))) := by
  simp only [assoc, swap, idK, tensor_ofFun, ofFun_comp_ofFun]
  rfl

/-- 'braid(X, I) = id(X)' up to the unitors. -/
theorem swap_unit_right [Fintype X] [DecidableEq X] :
  comp (swap X Unit) (leftUnitor X) = rightUnitor X := by

```

```
simp only [swap, leftUnitor, rightUnitor, ofFun_comp_ofFun]
rfl
```

Comonoid laws (Julia testsets “comonoid laws” and “coherence of copy and discard”).

Each is named after the Mathlib field it instantiates; see Theory/Correspondence.lean.

```
-- Coassociativity, 'ComonObj.comul_assoc': ' $\Delta \gg (X \triangleleft \Delta) = \Delta \gg (\Delta \triangleright X) \gg \alpha$ '. -/
theorem copy_assoc [Fintype X] [DecidableEq X] :
  comp (copy X) (tensor (idK X) (copy X)) =
    comp (comp (copy X) (tensor (copy X) (idK X))) (assoc X X X) := by
  simp only [copy, idK, assoc, tensor_ofFun, ofFun_comp_ofFun]
  rfl

-- Left counit, 'ComonObj.counit_comul': ' $\Delta \gg (\varepsilon \triangleright X) = (\lambda\_ X).inv$ '. -/
theorem copy_discard_left [Fintype X] [DecidableEq X] :
  comp (copy X) (tensor (discard X) (idK X)) = leftUnitorInv X := by
  simp only [copy, idK, discard_eq_ofFun, leftUnitorInv, tensor_ofFun, ofFun_comp_ofFun]
  rfl

-- Right counit, 'ComonObj.comul_counit': ' $\Delta \gg (X \triangleleft \varepsilon) = (\rho\_ X).inv$ '. -/
theorem copy_discard_right [Fintype X] [DecidableEq X] :
  comp (copy X) (tensor (idK X) (discard X)) = rightUnitorInv X := by
  simp only [copy, idK, discard_eq_ofFun, rightUnitorInv, tensor_ofFun, ofFun_comp_ofFun]
  rfl

-- The strict form of the counit laws checked in Julia: ' $\Delta(X) \cdot (\phi(X) \otimes id(X)) = id(X)$ '. -/
theorem copy_discard_left_strict [Fintype X] [DecidableEq X] :
  comp (comp (copy X) (tensor (discard X) (idK X))) (leftUnitor X) = idK X := by
  rw [copy_discard_left, leftUnitorInv, leftUnitor, ofFun_comp_ofFun, idK]
  exact ofFun_congr fun x => by simp

-- ' $\Delta(X) \cdot (id(X) \otimes \phi(X)) = id(X)$ '. -/
theorem copy_discard_right_strict [Fintype X] [DecidableEq X] :
  comp (comp (copy X) (tensor (idK X) (discard X))) (rightUnitor X) = idK X := by
  rw [copy_discard_right, rightUnitorInv, rightUnitor, ofFun_comp_ofFun, idK]
  exact ofFun_congr fun x => by simp

-- Cocommutativity, 'IsCommComonObj.comul_comm': ' $\Delta \gg \beta = \Delta$ '. -/
theorem copy_swap [Fintype X] [DecidableEq X] : comp (copy X) (swap X X) = copy X := by
  simp only [copy, swap, ofFun_comp_ofFun]
  rfl

-- Coherence of copy with the tensor, 'CopyDiscardCategory.copy_tensor':
' $\Delta[X \otimes Y] = (\Delta[X] \otimes \Delta[Y]) \gg \text{tensor}\mu$ ', Catlab's ' $\Delta(A \otimes B) = (\Delta(A) \otimes \Delta(B)) \cdot (id(A) \otimes \sigma(A,B) \otimes id(B))$ '. -/
← -/
theorem copy_prod [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y] :
  copy (X × Y) = comp (tensor (copy X) (copy Y)) (tensorμ X Y) := by
  simp only [copy, tensorμ, tensor_ofFun, ofFun_comp_ofFun]
  rfl

-- Coherence of discard with the tensor, 'CopyDiscardCategory.discard_tensor':
' $\varepsilon[X \otimes Y] = (\varepsilon[X] \otimes \varepsilon[Y]) \gg (\lambda\_ I).hom$ ', Catlab's ' $\phi(A \otimes B) = \phi(A) \otimes \phi(B)$ '. -/
theorem discard_prod [Fintype X] [Fintype Y] :
  discard (X × Y) = comp (tensor (discard X) (discard Y)) unitMul := by
  funext x u
  simp [unitMul]

-- 'CopyDiscardCategory.copy_unit': ' $\Delta[I] = (\lambda\_ I).inv$ ', Catlab's ' $\Delta(munit()) = id(munit())$ '. -/
theorem copy_unit : copy Unit = leftUnitorInv Unit := rfl

-- 'CopyDiscardCategory.discard_unit': ' $\varepsilon[I] = \mathbb{1} I$ ', Catlab's ' $\phi(munit()) = id(munit())$ '. -/
theorem discard_unit : discard Unit = idK Unit := by
  funext x u; simp [idK]
```

```

/-- Every deterministic kernel is a comonoid homomorphism: it commutes with copy
('Deterministic.copy_natural', the equation the Julia test checks for 'deterministic(X, Y, f)'). -/
theorem ofFun_copy_natural [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y] (f : X → Y) :
  comp (ofFun f) (copy Y) = comp (copy X) (tensor (ofFun f) (ofFun f)) := by
  simp only [copy, tensor_ofFun, ofFun_comp_ofFun]
  rfl

/-- ... and with discard ('Deterministic.discard_natural'). -/
theorem ofFun_discard_natural [Fintype X] [Fintype Y] [DecidableEq Y] (f : X → Y) :
  comp (ofFun f) (discard Y) = discard X := by
  rw [comp_ofFun_left]; rfl

end Kernel

end FiniteKernelsProofs.Finite

```

FiniteKernelsProofs.Finite.Laws

```

import FiniteKernelsProofs.Finite.Kernel
import Mathlib.Tactic.NormNum
import Mathlib.Tactic.Linarith

```

The two facts that `FiniteKernels.jl` pins numerically in `test/test_laws.jl` and that separate a Markov category from a cartesian one, proved exactly for the finite model of `Finite/Kernel.lean`:

- **Discard naturality iff normalised** (`comp_discard_eq_discard_iff`): the Markov axiom $f \cdot \diamond(B) == \diamond(A)$ holds for a table precisely when it is normalised. Julia testset “discard naturality iff normalised”, including the $[0.5 \ 0.5; \ 1.0 \ 0.5]$ counterexample (`comp_discard_apply` gives the column sums $[1.5, \ 1.0]$).
- **Copy naturality iff deterministic** (`comp_copy_eq_iff_isDeterministic`): for a normalised kernel, $f \cdot \Delta(B) == \Delta(A) \cdot (f \otimes f)$ holds iff every column is a point mass. Julia: “copy is natural for deterministic kernels only”.
- **Copy versus tensor** (`comp_copy_eq_tensor_iff_isPointMass`): the special case of a state $p : I \rightarrow X$; $\text{compose}(p, \text{mcopy}(X)) = \text{otimes}(p, p)$ (modulo $I \otimes I \cong I$, which is `copy Unit`) iff p is a point mass. Julia testset “copy once is not two samples (SPEC section 6)”.

The one piece of real arithmetic is `exists_pointMass_of_mul`: if $q \ x \ * \ q \ x' = [x = x'] \ q \ x$ and $\sum q = 1$, then q is an indicator. From $q \ x \ * \ q \ x = q \ x$ one gets $q \ x \in \{0, 1\}$; the sum forces at least one 1; and $q \ x \ * \ q \ a = 0$ for $x \neq a$ forces the rest to be 0.

```

namespace FiniteKernelsProofs.Finite.Kernel

open Finset

variable {X Y : Type}

```

Discard naturality iff normalised

```

/-- 'compose(k, delete(Y))' is the table of column sums. -/
theorem comp_discard_apply [Fintype Y] (k : Kernel X Y) (x : X) (u : Unit) :
  comp k (discard Y) x u =  $\sum y, k \ x \ y$  := by
  simp

/-- The Markov axiom 'f ·  $\diamond(B) == \diamond(A)$ ' holds for a table iff the table is normalised. -/
theorem comp_discard_eq_discard_iff [Fintype Y] (k : Kernel X Y) :
  comp k (discard Y) = discard X  $\Leftrightarrow$  Normalised k := by
  constructor
  • intro h x
    have := congrFun (congrFun h x) ()
    simp using this
  • intro h
    funext x u

```

```

simp [h x]

/-- Discard is natural for normalised kernels ('MarkovCategory.discard_natural'). -/
theorem Normalised.comp_discard [Fintype Y] {k : Kernel X Y} (hk : Normalised k) :
  Kernel.comp k (Kernel.discard Y) = Kernel.discard X :=
  (comp_discard_eq_discard_iff k).mpr hk

/-- A normalised state has total mass one ('state_discard' in 'BayesianNetworksProofs'). -/
theorem Normalised.state_discard [Fintype X] {p : State X} (hp : Normalised p) :
  Kernel.comp p (Kernel.discard X) = Kernel.idK Unit := by
  rw [hp.comp_discard, discard_unit]

```

The indicator lemma

```

/-- If 'q x * q x' = [x = x'] q x' and 'Σ q = 1' then 'q' is the indicator of a single point. -/
theorem exists_pointMass_of_mul [Fintype X] [DecidableEq X] (q : X → ℝ) (hsum : Σ x, q x = 1)
  (h : ∀ x x', q x * q x' = if x = x' then q x else 0) :
  ∃ a, ∀ x, q x = if x = a then 1 else 0 := by
  have h01 : ∀ x, q x = 0 ∨ q x = 1 := by
    intro x
    have hx := h x x
    rw [if_pos rfl] at hx
    have : q x * (q x - 1) = 0 := by rw [mul_sub, mul_one, hx, sub_self]
    rcases mul_eq_zero.mp this with h0 | h1
    · exact Or.inl h0
    · exact Or.inr (sub_eq_zero.mp h1)
  have hex : ∃ a, q a = 1 := by
    by_contra hne
    push Not at hne
    have hzero : ∀ x, q x = 0 := fun x => (h01 x).resolve_right (hne x)
    have : Σ x, q x = 0 := Finset.sum_eq_zero fun x _ => hzero x
    rw [this] at hsum
    exact zero_ne_one hsum
  obtain ⟨a, ha⟩ := hex
  refine ⟨a, fun x => ?_⟩
  by_cases hxa : x = a
  · subst hxa; simp [ha]
  · have hx := h x a
    rw [if_neg hxa, ha, mul_one] at hx
    simp [hxa, hx]

/-- Conversely, an indicator satisfies the product identity. -/
theorem mul_eq_of_pointMass [DecidableEq X] (a x x' : X) :
  (if x = a then (1 : ℝ) else 0) * (if x' = a then 1 else 0) =
  if x = x' then (if x = a then 1 else 0) else 0 := by
  by_cases hx : x = a
  · subst hx
    by_cases hx' : x' = x
    · subst hx'; simp
    · simp [hx', Ne.symm hx']
  · by_cases hx' : x' = a
    · subst hx'; simp [hx]
    · simp [hx, hx']

```

Copy naturality iff deterministic

```

/-- 'compose(k, mcopy(Y))' puts the column of 'k' on the diagonal (draw once, then copy). -/
theorem comp_copy_apply [Fintype Y] [DecidableEq Y] (k : Kernel X Y) (x : X) (y1 y2 : Y) :
  comp k (copy Y) x (y1, y2) = if y1 = y2 then k x y1 else 0 := by
  simp only [comp_apply, copy, ofFun_apply, Prod.mk.injEq, mul_ite, mul_one, mul_zero]
  by_cases h : y1 = y2
  · subst h
    simp

```

```

• rw [if_neg h]
  apply Finset.sum_eq_zero
  intro y _
  rw [if_neg]
  rintro ⟨rfl, rfl⟩
  exact h rfl

/-- 'compose(mcopy(X), otimes(k, k))' is the outer product of the column with itself
(draw twice, independently). -/
theorem copy_comp_tensor_apply [Fintype X] [DecidableEq X] (k : Kernel X Y) (x : X) (y₁ y₂ : Y) :
  comp (copy X) (tensor k k) x (y₁, y₂) = k x y₁ * k x y₂ := by
  simp [copy]

/-- For a normalised kernel, 'f · Δ(B) == Δ(A) · (f ⊗ f)' holds iff 'f' is deterministic. -/
theorem comp_copy_eq_iff_isDeterministic [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y]
  (k : Kernel X Y) (hk : Normalised k) :
  comp k (copy Y) = comp (copy X) (tensor k k) ↔ IsDeterministic k := by
  constructor
  • intro h x
    refine exists_pointMass_of_mul (k x) (hk x) fun y₁ y₂ => ?_
    have := congrFun (congrFun h x) (y₁, y₂)
    rw [comp_copy_apply, copy_comp_tensor_apply] at this
    exact this.symm
  • intro h
    funext x y
    rcases y with ⟨y₁, y₂⟩
    obtain ⟨a, ha⟩ := h x
    rw [comp_copy_apply, copy_comp_tensor_apply, ha y₁, ha y₂]
    exact (mul_ite_eq_of_pointMass a y₁ y₂).symm

/-- Deterministic kernels are exactly the 'ofFun f' (Julia 'deterministic(X, Y, f)'). -/
theorem isDeterministic_iff_exists_ofFun [DecidableEq Y] (k : Kernel X Y) :
  IsDeterministic k ↔ ∃ f : X → Y, k = ofFun f := by
  constructor
  • intro h
    choose f hf using h
    exact ⟨f, funext fun x => funext fun y => hf x y⟩
  • rintro ⟨f, rfl⟩ x
    exact ⟨f x, fun _ => rfl⟩

/-- Deterministic kernels are normalised. -/
theorem IsDeterministic.normalised [Fintype Y] [DecidableEq Y] {k : Kernel X Y}
  (h : IsDeterministic k) : Normalised k := by
  obtain ⟨f, rfl⟩ := (isDeterministic_iff_exists_ofFun k).mp h
  exact Normalised.ofFun f

```

Copy versus tensor for states

```

theorem isPointMass_iff_isDeterministic [DecidableEq X] (p : State X) :
  IsPointMass p ↔ IsDeterministic p := by
  constructor
  • rintro ⟨a, ha⟩ _; exact ⟨a, ha⟩
  • intro h; exact h ()

theorem isPointMass_iff_exists_pointMass [DecidableEq X] (p : State X) :
  IsPointMass p ↔ ∃ a, p = pointMass a := by
  constructor
  • rintro ⟨a, ha⟩
    exact ⟨a, funext fun u => funext fun x => by cases u; exact ha x⟩
  • rintro ⟨a, rfl⟩
    exact ⟨a, fun x => rfl⟩

/-- **Copy once is not two samples.** For a normalised state 'p', drawing once and copying
('compose(p, mcopy(X))') equals drawing twice independently ('otimes(p, p)'), with the domain
'I ⊗ I' identified with 'I' by 'copy Unit') iff 'p' is a point mass. -/

```

```

theorem comp_copy_eq_tensor_iff_isPointMass [Fintype X] [DecidableEq X] (p : State X)
  (hp : Normalised p) :
  comp p (copy X) = comp (copy Unit) (tensor p p) ↔ IsPointMass p := by
  rw [comp_copy_eq_iff_isDeterministic p hp, isPointMass_iff_isDeterministic]

/-- The same statement entrywise, without the 'I ⊗ I ≅ I' reindexing:
'copied.table == independent.table' iff 'p' is a point mass. -/
theorem comp_copy_eq_tensor_iff_isPointMass' [Fintype X] [DecidableEq X] (p : State X)
  (hp : Normalised p) :
  (∀ x x', comp p (copy X) () (x, x') = tensor p p ((), ()) (x, x')) ↔ IsPointMass p := by
  rw [← comp_copy_eq_tensor_iff_isPointMass p hp]
  constructor
  · intro h
    funext u y
    rcases y with ⟨x, x'⟩
    cases u
    rw [copy_comp_tensor_apply]
    exact h x x'
  · intro h x x'
    have := congrFun (congrFun h ()) (x, x')
    rw [copy_comp_tensor_apply] at this
    exact this

/-- Point masses do satisfy the equation (Julia: 'compose(q, mcopy(X)) ≈ otimes(q, q)' for
'q = point_mass(X, l)'). -/
theorem pointMass_comp_copy [Fintype X] [DecidableEq X] (a : X) :
  comp (pointMass a) (copy X) = comp (copy Unit) (tensor (pointMass a) (pointMass a)) :=
  (comp_copy_eq_tensor_iff_isPointMass _ (Normalised.pointMass a)).mpr ⟨a, fun _ => rfl⟩

/-- The uniform state on two points is not a point mass, so for it the equation fails
(the Julia 'state(X, [0.2, 0.3, 0.5])' example, in its smallest form). -/
example : ¬ IsPointMass (fun _ => (1 / 2 : ℝ) : State (Fin 2)) := by
  rintro ⟨a, ha⟩
  have := ha a
  norm_num at this

example :
  comp (fun _ => (1 / 2 : ℝ) : State (Fin 2)) (copy (Fin 2)) ≠
  comp (copy Unit) (tensor (fun _ => (1 / 2 : ℝ)) (fun _ => (1 / 2 : ℝ))) := by
  rw [Ne, comp_copy_eq_tensor_iff_isPointMass _ (fun _ => by simp)]
  rintro ⟨a, ha⟩
  have := ha a
  norm_num at this

end FiniteKernelsProofs.Finite.Kernel

```

FiniteKernelsProofs.Theory.Correspondence

```

import Mathlib.CategoryTheory.CopyDiscardCategory.Basic
import Mathlib.CategoryTheory.CopyDiscardCategory.Deterministic
import Mathlib.CategoryTheory.MarkovCategory.Basic

```

Dictionary between the GATlab theories used by MarkovCategories.jl (src/theory.jl) and Mathlib's CopyDiscardCategory / MarkovCategory, with the theorem of the finite model (Finite/Kernel.lean, Finite/Laws.lean) that instantiates each axiom.

This module spans two Julia packages: the finite model it instantiates the axioms in belongs to FiniteKernels.jl, which holds these proofs, while the GATlab side of the dictionary now lives in MarkovCategories.jl (its src/theory.jl and src/finstoch_model.jl); FiniteKernels.jl itself carries no Catlab or GATlab dependency and names the same operations compose_kernel, tensor_kernel, identity_kernel, copy_kernel, discard_kernel and swap_kernel.

ThCopyDiscardCategory is MarkovCategories.jl's alias for Catlab's ThMonoidalCategoryWithDiagonals

(Catlab/src/theories/Monoidal.jl, “Cartesian category” section); ThMarkovCategory adds one axiom, the naturality of delete.

GATlab (ThMonoidalCategoryWithDiagonals)	Mathlib	Finite model (Kernel.*)
$\text{mcopy}(A) :: A \rightarrow A \otimes A, \Delta$	<code>ComonObj.comul, $\Delta[X]$</code>	<code>copy X</code>
$\text{delete}(A) :: A \rightarrow \text{munit}(), \diamond$	<code>ComonObj.counit, $\epsilon[X]$</code>	<code>discard X</code>
$\Delta(A) \cdot (\Delta(A) \otimes \text{id}(A)) == \Delta(A) \cdot (\text{id}(A) \otimes \Delta(A))$	<code>ComonObj.comul_assoc</code>	<code>copy_assoc</code>
$\Delta(A) \cdot (\diamond(A) \otimes \text{id}(A)) == \text{id}(A)$	<code>ComonObj.counit_comul</code>	<code>copy_discard_left(_strict)</code>
$\Delta(A) \cdot (\text{id}(A) \otimes \diamond(A)) == \text{id}(A)$	<code>ComonObj.comul_counit</code>	<code>copy_discard_right(_strict)</code>
$\Delta(A) \cdot \sigma(A, A) == \Delta(A)$	<code>IsCommComonObj.comul_comm</code>	<code>copy_swap</code>
$\Delta(A \otimes B) == (\Delta(A) \otimes \Delta(B)) \cdot (\text{id}(A) \otimes \sigma(A, B) \otimes \text{id}(B))$	<code>CopyDiscardCategory.copy_tensorprod</code>	<code>copy_tensorprod</code>
$\diamond(A \otimes B) == \diamond(A) \otimes \diamond(B)$	<code>CopyDiscardCategory.discard_tensorprod</code>	<code>discard_tensorprod</code>
$\Delta(\text{munit}()) == \text{id}(\text{munit}())$	<code>CopyDiscardCategory.copy_unit</code>	<code>copy_unit</code>
$\diamond(\text{munit}()) == \text{id}(\text{munit}())$	<code>CopyDiscardCategory.discard_unit</code>	<code>discard_unit</code>
symmetric monoidal structure (ThSymmetricMonoidalCategory)	<code>SymmetricCategory (extended)</code>	<code>comp_tensor, tensor_assoc, tensor_unit_*, swap_swap, tensor_swap, hexagon</code>

GATlab (ThMarkovCategory)	Mathlib	Finite model
$f \cdot \diamond(B) == \diamond(A) \dashv [f :: (A \rightarrow B)]$	<code>MarkovCategory.discard_natural</code>	<code>comp_discard_eq_discard_iff</code> (holds iff Normalised)
(not an axiom) $f \cdot \Delta(B) == \Delta(A) \cdot (f \otimes f)$	<code>Deterministic f (= IsComonHom f)</code>	<code>comp_copy_eq_iff_isDeterministic</code>

Catlab’s tensor is strict, so the Mathlib associator, unitors and tensorµ are identities on the Julia side; in the finite model they are the reindexing kernels `assoc`, `leftUnitor`, ..., `tensorµ`. The generic consequences of these axioms (`discard_natural` as a theorem, `deterministic_comp`, `deterministic_copy`, `state_discard`) are stated once for the abstract Mathlib classes in `BayesianNetworks.jl/proofs/BayesianNetworksProofs/Markov/Basic.lean` and are not repeated here. The examples below only check that each Mathlib field has the type the table claims.

```
namespace FiniteKernelsProofs.Theory

open CategoryTheory MonoidalCategory ComonObj CopyDiscardCategory

universe v u

variable {C : Type u} [Category.{v} C] [MonoidalCategory.{v} C]

section CopyDiscard

variable [CopyDiscardCategory C]

-- Commutative comonoid axioms of 'ThMonoidalCategoryWithDiagonals'.
example (X : C) :  $\Delta[X] \gg (X \triangleleft \Delta[X]) = \Delta[X] \gg (\Delta[X] \triangleright X) \gg (\alpha_- X X X).hom := \text{comul\_assoc } X$ 
example (X : C) :  $\Delta[X] \gg (\epsilon[X] \triangleright X) = (\lambda_- X).inv := \text{counit\_comul } X$ 
example (X : C) :  $\Delta[X] \gg (X \triangleleft \epsilon[X]) = (\rho_- X).inv := \text{comul\_counit } X$ 
example (X : C) :  $\Delta[X] \gg (\beta_- X X).hom = \Delta[X] := \text{IsCommComonObj.comul\_comm } X$ 

-- Coherence axioms.
-- (' $\Delta[X \otimes Y]$ ' must be read with the 'CopyDiscardCategory.comonObj' instance, not the generic
-- tensor product of comonoids 'Comon.instComonObjTensorObj'; the axiom says the two agree.)
example (X Y : C) :
  letI : ComonObj (X  $\otimes$  Y) := CopyDiscardCategory.comonObj (X  $\otimes$  Y)
   $\Delta[X \otimes Y] = (\Delta[X] \otimes_m \Delta[Y]) \gg \text{tensor}\mu \ X \ X \ Y \ Y := \text{copy\_tensor } X \ Y$ 
example (X Y : C) :
```

```

    let I : ComonObj (X ⊗ Y) := CopyDiscardCategory.comonObj (X ⊗ Y)
    ε[X ⊗ Y] = (ε[X] ⊗m ε[Y]) > (λ_ (1_ C)).hom := discard_tensor X Y
example : Δ[1_ C] = (λ_ (1_ C)).inv := copy_unit
example : ε[1_ C] = 1 (1_ C) := discard_unit

-- Copy naturality is not an axiom; it is the definition of 'Deterministic'.
example {X Y : C} (f : X → Y) [Deterministic f] : f > Δ[Y] = Δ[X] > (f ⊗m f) :=
  Deterministic.copy_natural f

end CopyDiscard

section Markov

variable [MarkovCategory C]

-- The one axiom 'ThMarkovCategory' adds.
example {X Y : C} (f : X → Y) : f > ε[Y] = ε[X] := MarkovCategory.discard_natural f

end Markov

end FiniteKernelsProofs.Theory

```

Roadmap (contains sorry)

```

import FiniteKernelsProofs.Finite.Laws
import Mathlib.CategoryTheory.MarkovCategory.Basic

```

Module `FiniteKernelsProofs.Roadmap`. Statements that are **not** part of the default build and are excluded from `Audit.lean` (make `roadmap` builds this file on its own). Everything here that is sorry-free is a candidate to move into the default target once it is useful to something downstream.

- `FinStoch` with its `Category` instance (stochastic kernels between finite types) is proved from `Finite/Kernel.lean` and is sorry-free.
- The `MonoidalCategory` and `MarkovCategory` instances are deferred (plan “Lean 4 layer”: do not vendor the open `Mathlib` PR that adds `Stoch`; a `MarkovCategory` instance on finite types is only worth building when a downstream statement needs it). The ingredients are all in `Finite/Kernel.lean` (`comp_tensor`, `tensor_assoc`, `tensor_unit_*`, `hexagon`, `copy_assoc`, `copy_discard_*`, `copy_swap`, `copy_prod`, `discard_prod`, `copy_unit`, `discard_unit`, `Normalised.comp_discard`); what is missing is the bookkeeping of whiskerings, the pentagon and triangle identities, and the `Subtype` plumbing.

```

namespace FiniteKernelsProofs.Roadmap

open CategoryTheory FiniteKernelsProofs.Finite

/-- Objects of FinStoch: finite types with decidable equality ('FiniteSpace'). -/
structure FinStoch where
  /- The set of joint states. -/
  carrier : Type
  [fintype : Fintype carrier]
  [decEq : DecidableEq carrier]

attribute [instance] FinStoch.fintype FinStoch.decEq

/-- Morphisms are stochastic (nonnegative, normalised) kernels; composition is 'compose'. -/
instance : Category FinStoch where
  Hom X Y := {k : Kernel X.carrier Y.carrier // Kernel.Stochastic k}
  id X := ⟨Kernel.idK X.carrier, Kernel.Stochastic.ofFun id⟩
  comp k l := ⟨Kernel.comp k.1 l.1, k.2.comp l.2⟩
  id_comp k := Subtype.ext (Kernel.idK_comp k.1)
  comp_id k := Subtype.ext (Kernel.comp_idK k.1)
  assoc k l m := Subtype.ext (Kernel.comp_assoc k.1 l.1 m.1)

/-- Deferred: the symmetric monoidal structure with 'otimes' = 'Kernel.tensor'. -/

```

```
noncomputable instance : MonoidalCategory FinStoch := sorry

/-- Deferred: the Markov structure with 'mcopy' = 'Kernel.copy', 'delete' = 'Kernel.discard';
'discard_natural' would be 'Kernel.Normalised.comp_discard'. -/
noncomputable instance : MarkovCategory FinStoch := sorry

end FiniteKernelsProofs.Roadmap
```